SOFT KOMPJUTING

- **Nadgledano vs nenadgledano ucenje:**

Osnovna razlika je u tome kako se modeli treniraju i koje trening podatke algoritmi koriste. Koje trening podatke koriste: nadgledano koristi labelirane dok nenadgledano nelabelirane. Nadgledano ima info kako tacan rezultat treba da izgleda.

Nenadgledanom ucenju samo das nelabelirane podatke i on sam pronalazi sablone. Nenadgledano se koristi za istrazivacku analizu podataka i klasterovanje.

Nadgledano uci odnose izmedju ulaznih i izlaznih podataka, koriste se za predikciju ondnosno zadatke klasifikacije i regresije.

- **Hard computing vs Soft computing**

Hard Computing = ori = algoritmi pretrage, algoritmi sortiranja, deterministicki, tacno definisan ulaz, tacna rjesenja

Soft Computing = nedeterministicki, ulaz dvosmislen i zasumljen, stohasticko, priblizna rjesenja

- **Digitalna reprezentacija slika**

Predstavljanje slika u takvoj formi da omogucava masinama da interpretiraju i razumiju vizuelne informacije preko raznih algoritama. (Slike se reprezentuju kao matrice brojeva, gdje je glavni element piksel – ono na cemu smo mi radili).

2 vrste diskretizacije digitalne slike: prostorna i broj nijansi. Prostorna: elementi digitalne slike zovu se pikseli. Broj nijansi: vrednost svakog piksela je najcese od 0 do 255 (raspon od 256 vrijednosti) odnosno 8 bita.

Kvalitet slike zavisi od oba tipa diskretizacije: 1. prostorna: rezolucija slike (proizvod broja piksela po koloni i redu). Veca rezolucija = bolji kvalitet slike. 2. broj nijansi: kvantizacija.

2 vrste reprezentacije digitalne slike: rasterski i vektorski. Bla bla

Podjela u odnosu na podatke unutar piksela (kvantizacija?): binarne, nijanse sive, multispektralna slika. Binarna: 0 i 1, izdvajanje foreground-a od background-a tj segmentacija. Grayscale: monohromatske: imaju 2 dimenzije (kao i prethodna) tj piksel je predstavljen jednom vrijednoscu. Multispectral: piksel je predstavljen sa 3 vrijednosti tj svaka komponenta boje ima svoj kanal. Modeli boja (aditivni i supstraktivni.....grafika): rgb i hsv.

Kako konvertovati rgb u grayscale? 1. Srednja vrijednost rgb komponenti 2. metod perceptivne osvjetljenosti (najvise vidimo zelenu pa crvenu pa plavu boju i mnozimo tim postocima: 60%, 30% i 10% respektino).

- **Histogram slike**

Histogram slike je grafički prikaz distribucije intenziteta piksela u datoj slici. Na x-osi se obično nalaze vrednosti intenziteta piksela (npr. od 0 do 255 ali je obicno podijeljeno u barove), dok se na y-osi prikazuje koliko ima piksela sa tim intenzitetom. Najcesce su predstavljeni u barovima gdje jedan bar obuhvata opseg od npr 20 piksela. Za slike u boji imacemo predstavu za svaki kanal posebno.

Histogram pruža informacije o:

1. Osvetljenju: min i max nam govore o rasponu intenziteta koji je snimljen, a prosjecna vrijednost govori o osvjetljenu slike. Npr ako su vrednosti hstograma skoncentirane uglavnom na lijevoj strani onda je nisko osvjetljene, a na desnoj onda je visoko.
2. Kontrastu: ako su vrijednosti skoncentrisane oko jednog odredjenog intenziteta (kao prethodni primjeri) -> los kontrast u osvjetljenu. Ako

su vrijednosti ravnomjerno raspodjeljene po intenzitetima -> dobar kontrast u osvjetljenju.

Cemu ovo sluzi? Mozemo da uradimo poravnjanje histograma i time pojacamo kontrast na slici (ako je slika overexposed ili underexposed).

Histogram ne cuva podatke o prostornoj povezanosti piksela na slici.

- **Operacije nad slikama, susjedstvo piksela**

Opreacije: sabiranje (za uklanjanje suma, za blendovanje), oduzimanje (za trazenje razlika tj detekciju pokreta, za video kompresiju), logicke operacije (nad binarnim slikama: za segmentaciju, kombinovanje slika, green screen, negativ), afine transformacije.

Susjedstvo piksela(spominje se u okviru logickih operacija)=za definiciju prostorne povezanosti: 4-connectivity, 8-connectivity.

Uprosečavanje: vrednost piksela zamenjujemo prosečnom vrednošću njegovog susedstva (zavisi od definicije susedstva) -> uklanjamo šum, ali dobijamo zamućeniju sliku.

- **Segmentacija slike**

Postupak podjele slike na regione sa slicnim karakterisitkama. Izdvajanje foreground-a od background-a tj ekstrakcija bitnih informacija. Cilj segmentacije je da se pojednostavi ili promeni predstava slike u nešto što bi bilo smislenije i lakše za analizu. Segmentaciju mozemo vrsiti po: boji, osvjetljenosti, ivicama... Segmentacija je najčešće kritičan korak u analizi slike jer je to tačka u kojoj se prelazi sa piksela na objekat kao jedinice obrade.

Podjela segmentacije: preko praga/histogram(otsu, lokalni threshold), klasterovanje(k-means, superpixels, watershed), pracenje ivica(HOG)

Najjednostavniji način za segmentaciju slike je prvo(preko praga) tj thresholding -> poredimo osvetljenost piksela sa pragom (ili više pragova) i

svrstavamo piksele u dve (ili više) kategorija. Postupak... (od grayscale pa izlaz).

- **OTSU**

Otsu's metoda je tehnika automatskog određivanja praga za pretvaranje slike u crno-belu. Krajnji rezultat je binarna slika sa jasno definisanim objektima i pozadinom, a postupak se odvija automatski, bez potrebe za ručnim podešavanjem praga. gleda histogram kako bi odredio prag -> prag je u “dolini” histograma. Problem: sum (moze donekle da se resi blur-om), zanemaruje prostorne relacije. Ne moze se koristiti kod nejednakog osvjetljenja.

Rjesenje nejednakog osvjetljenja: trazimo lokalni prag, na osnovu nekih povezanih regiona tj definicije susjedstva.

- **Morfoloske operacije**

Gotovo uvek će rezultujuće slike segmentacije imati neke probleme u vidu crnog ili belog šuma(negdje dodaje vise objekata, a neke spaja u jedan). Da bi se rijesio prethodni problem, rade se morfoloske operacije. Opsti algoritam: ulaz (binarna slika i strukturni element), izlaz (binarna slika iste velcinine kao ulazna). Strukurni element: matrica sastavljenu od 0 i 1, definiše okolinu piksela. Bazira se na OBLICIMA.

Osnovne binarne operacije su: • Dilacija (logicko ili)– dodaje piksele na ivicu objekata na slici • Erozija (logicko i) – uklanja piskele sa ivica objekata na slici.

Njihovom kombinacijom možemo konstruisati operatore za: • Otvaranje (erozija pa dilacija)- uklanjanje suma bez promjene velicine objekata • Zatvaranje – popunjavanje bez promjene velicine objekta • Detekciju ivica: dilacija – erozija i obrnuto

- **Konvolucija**

Konvolucija je matematička operacija definisana nad dve funkcije koja proizvodi treću funkciju. Funkcije = slike. Ulazna slika i filter daju izlaznu sliku. „Prevlačimo“ kernel preko svakog piksela na slici, Vrednost piksela na izlaznoj slici dobijamo tako što množimo odgovarajuće vrednosti ulazne slike i kernela, a zatim sve umnoške saberemo

- **K-means**

Načini inicijalizacije centroida, različiti kriterijumi stopiranja, određivanje korektnog broja klastera (k) za datu sliku. Problem: koristi samo boju, ne i lokaciju, pa su izdvojene oblasti obicno diskonektovane, a zeleli bismo malo više prostorne koherentnosti.

- **Superpixels**

Modifikacija k-means koja ukljucuje i prostornu povezanost piksela. Sliku od piksela predstavicemo manjim brojem superpiksela (u odnosu na broj piksela) -> kompaktnija predstava slike. Dakle, superpixeli su kompaktne i homogene regije u digitalnoj slici koje grupišu susedne piksele sa sličnim karakteristikama, kao što su boja, tekstura ili intenzitet. Cilj superpiksela je smanjivanje dimenzionalnosti slike, olakšavanje analize i obrade podataka. Različite metode generišu superpixele na osnovu različitih kriterijuma, a neki od popularnih algoritama uključuju Simple Linear Iterative Clustering (SLIC): slicno kao k-means. Broj superpiksela: podešavanje prema tome koliko želimo da superpikseli prate ivice

- **Segmentacija pomocu rasta regiona = Region growing**

...

- **Watershed (razvodje/sliv)**

Tipično se koristi za rešavanje jedne od najtežih operacija u procesiranju slike –> odvajanje sličnih objekata koji se dodiruju. Metafora naziva (razvodje/sliv) inspirisana je fizičkim konceptom sliva, pa se slika intepretira kao 3D površina sa „dolinama“(lokalni min) i „planinama“(gradijenti). U dolinama nalaze se minimumi regije, a izmedju njih brane da odvojimo razlicite regije. (minimumi suplji, pustamo vodu, kako bi se izbeglo da se voda koja dolazi od dva različita minimuma izmeša, pravi se „brana“ kadgod bi se desilo spajanje vodenih površina. Konačno, jedina stvar koja je vidljiva na površini bi bile ove „brane“). Ove „brane“ se zovu watershed lines i njih zelimo da detektujemo (granice izmedju dva objekta).

Markeri su od suštinskog značaja za vođenje procesa segmentacije. Markeri se obično biraju na osnovu određenih kriterijuma, kao što su intenzitet ili vrednosti gradijenta. Ovi markeri pomažu u identifikaciji regiona od interesa.

Watershed se primenjuje na sliku gradijenta, tretirajući sliku kao topografsku površinu. Vrednosti gradijenta na slici se tumače kao nadmorske visine. Simulira se plavljenje ove površine sa markera, sa branama koji se formiraju oko svakog markera.

Problem: Slika može biti prekomerno segmentirana usled šuma (previse lokalnih min). Rešenje: ograničiti broj dozvoljenih regija -> koristimo predefinisan skup markera, gdje treba da definisemo interne markere (siguran foreground, okruzeni tackama “vece nadmorske visine”) i eksterne markere (siguran background)

- **Segmentacija pomocu razdvajanja i spajanja regiona = split-merge, pomocu granica regiona, pomocu grafova**

...

- **Detekcija ivica**

Detekcija ivica je proces identifikacije prelaza između različitih regiona intenziteta na slici. Različite metode detekcije ivica koriste različite tehnike, uključujući primenu filtera, gradijentnih operatora i analizu teksture. Ovaj proces često se koristi kao prvi korak u analizi i prepoznavanju oblika. Detekcija ivica pomaže u izdvajanju bitnih informacija i smanjenju dimenzionalnosti slike. Popularni algoritmi u ovom kontekstu uključuju Canny detektor ivica, Sobel operator, Robert-Cross...

Cilj: konvertovati 2D sliku u skup krivih. Kako se to postize?

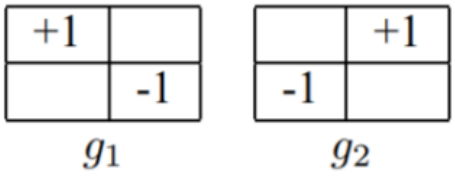
Slike predstavimo kao funkciju intenziteta piksela. Pri detekciji se oslanjamo samo na intenzitet – ovo nije savršeno.(Ako bi se dva objekta istog intenziteta preklapala, ovo bi nas sprečilo da detektujemo ivicu. Međutim, veoma retko se dešava da su dva objekta na slici baš istog intenziteta, pa je intenzitet dobar indikator ivica.)

Ivica je mesto brze promjene funkcije intenziteta. Dobra indikacija ivice (nagle promene intenziteta piksela) jeste izvod funkcije intenziteta. Koristi se gradijent umjesto izvoda posto je slika 2D. Gradijent slike ukazuje na smer najveće promene u intenzitetu piksela. Magnituda gradijenta je “snaga” ivice.

Kada primjenimo diferencijalni operator(kernel) na sliku (sa ciljem dobijanja fje izvoda odnosno fje gradijenta te slike) dobijamo kao rezultat 0 tamo gdje nema ivice, a veliku vrijednost na mjestima promjene gradijenta.

- **Roberts Cross**

Za detekciju ivica, koriste se 2 filtera, jedan filter za izvod po jednoj dijagonali, drugi za izvod po drugoj, pa se rezultati onda kombinuju da se pronadje magnituda izvoda. Kerneli su jednaki, samo zarotirani za 90 stepeni :



Prednosti: brz, koristimo samo 4 piksela ulaza i od operacija samo + i -, nema parametara za podesavanje. Mane: osjetljiv na sum jer posmatra razliku na jednom pikelu, nejasno koji izlazni piksel odgovara kojem ulaznom, slab odziv na neostre ivice.

- **Sobel**

Mane: sporiji od Robert Cross

Prednosti: veci kernel pa je manje osjetljiv na sum

Ivice su vece zbog efekta izgladjivanja

- **Canny**

Canny detektor ivica je optimalan detektor ivica. Vrsi i dobru detekciju (reaguje na ivice a ne na sum) i dobru lokalizaciju (detektovana ivica je blizu stvarne; odgovor je minimalan–> jedan piksel po ivici). Nagodba izmedju detekcije i lokalizacije: više izglađivanja pobojšava detekciju, ali lokalizaciju čini gorom.

On radi tako što:

1. "zaglađuje" sliku = uklanja sum gausovim operatorom

2. pronalazi gde su promene intenziteta najveće = racunjanje magnintude i orijentacije gradijenta (slicno kao u Robert Cross)

3. tanji te ivice = non-maxima suppression. Algoritam ide po grebenima (ridge tracker) i postavlja na 0 sve piksele koji nisu na vrhu grebena: Pronalazi se piksel maksimalne vrednosti na ivici. Piksel □ ima veći intenzitet od piksela □ i □ koji imaju istu orijentaciju (ugao) gradijenta, □ čuvamo, a □ i □ uklanjamo.

4. na kraju odlučuje koje ivice su relevantne tako sto koristi 2 praga - sve iznad jednog sigurno jeste, a sve ispod drugog sigurno nije ivica i onda za ovo izmedju gleda da li je povezano sa sigurnim ivicama.

Tri parametra: širina Gausovog kernela, □1 i □2

Problem: Y-junctions = kad se 3 ivice sastanu na jednom mjestu (u obliku Y) moze odvojiti jednu ivicu

- **Hough transformacija:**

Možemo je koristiti za detekciju proizvoljnih oblika – ako možemo taj oblik da predstavimo u matematičkoj formi: Detekcija linija, Detekcija krugova, Ostale parametarske krive (proizvoljni oblici). Adresira ovaj problem grupisanja tačaka ivice u objekte. Algoritam:

1. detektovati ivice jer Hough transformacija kao ulaz uzima binarnu sliku (onda se detekcija ivica vrsi prije hough transform algoritma)

2. transformiše PIKSELE SA IVICE u parametarski prostor (prostor svih mogucih k i n, od kojih jedna kombinacija jedinstveno predstavlja pravu – za slucaj prave) i snimanje u akumulator. Dimenzionalnost hough prostora zavisi od broja nepoznatih parametara (u slucaju prave je dakle 2). Hough prostor je beskonačan – podelićemo ga u konačne segmente (akumulatorske ćelije). Na koliko akumulatora podijeliti tj koliki da nam bude broj

Vrijednost svake celije se uveca za svaku liniju Hough prostora koja ju presjeca.

3. Interpretacija akumulatora (pronalaženje beskonačnih linija): celije koje imaju dovoljan broj glasova (iznad nekog praka) reprezentuju odgovarajuce linije

4. konverzija beskonacnih linija u konacne

- **HOG**

Za ekstrakciju obiljezja iz slike tj smanjenje njene dimenzionalnosti (sa sirina*visina*3 na vektor duzine n). Deep learning inace preskace ovaj korak ekstrakcije obiljezja. Ono sto HOG uradi koristice klasifikatori poput SVM.

Ostalo u svesci.

Napomena: vrsi se normalizacija histograma na nivou jednog bloka.

- **SVM = Support Vector Machines**

Nadgledano ucenje, za klasifikaciju i regresiju, na osnovu ekstrakovanih obiljezja tj kreiranog skupa.

Vektori obiljezja se tretiraju kao tacke u visokodimenzionalnom prostoru i nalazimo hiperravni koje ce razdvojiti jednu klasu od druge. Margina razdvajajuće hiperravni je minimum rastojanja od te hiperravni do neke od tačaka skupa podataka. Koja hiperravan je najbolja? Ona koja maksimizuje marginu izmedju podataka.

- **KNN = K najblizih susjeda**

Nadgledano ucenje, za klasifikaciju i regresiju. Kako funkcionise algoritam?

Ako imamo sliku i zelimo da je klasifikujemo KNN će odabrati K najbližih suseda te slike u prostoru karakteristika (na osnovu odabrane metrike), gde svaki sused predstavlja neku sliku iz trening skupa. Odluka o klasi slike zasniva se na većinskoj klasi među K najbližih suseda.

Hiper-parametri: Metrika rastojanja i K

Vreme treniranja: □(1), Vreme testiranja: (N) -> ovo je loše jer voleli bismo da naš klasifikator bude brz prilikom predikcije, a sporo treniranje bilo bi u redu.

Rijetko se koristi u praksi i poznat je kao “lenji” algoritam jer odlaze proces ucenja sve dok se predikcija ne zatrazi. Za predikciju potrebno je proci kroz citav skup podataka (izracunati rastojanje i odrediti susjede) te dugo traje.

- **Hiper-parametri**

Hiperparametri su parametri modela u mašinskom učenju koji se postavljaju pre nego što proces učenja počne. Oni se ne uče se iz podataka (to su bas parametri), već se postavljaju pre treniranja modela (mi ih zadajemo) i utiču na sam proces učenja. Hiperparametri se često podešavaju empirijski: isprobati šta radi najbolje tj. za svaku kombinaciju vrednosti hiper-parametra obučiti model i izmeriti njegove performanse. Odabrati one hiper-parametre koji rezultuju najboljim performansama. Ali na kojim podacima trenirati model a na kojim meriti performanse modela?

Podeliti dostupne podatke na:

1. trening skup – treniranje modela (odabir parametara)

2. validation (hold-out) skup – odabir hiper-parametara

3. test skup – evaluacija performansi modela

Dakle, parametre podesava sam algoritam, a hiper-parametre podesavamo mi!

Primjer parametara CNN: tezine, bias...

Primjer hiper-parametara CNN: broj slojeva, velicina filtera, stride, zero padding...

- **Detekcija objekata**

Detekcija objekata je proces prepoznavanja i lokalizacije različitih entiteta ili oblika na slici. Ova tehnika se koristi u računarskom vidu kako bi se identifikovali specifični objekti, kao što su lica, automobili ili drugi predmeti od interesa. Cilj je tačno označiti lokaciju i vrstu objekta na slici.

<u>Detekcija objekata pomoću HOG deskriptora:</u>

1. Sastaviti trening skup od P pozitivnih primera (slike koje sadrže objekat), N negativnih primera (slike koje ne sadrže objekat). Svaki primer (sliku) reprezentovati pomoću HOG deskriptora.
2. Trenirati model (npr. SVM) na trening skupu

Ali trenirani model može da prepozna samo objekte na isečcima fiksne veličine. Stoga, selektovaćemo podregije (isečke) originalne slike i primeniti trenirani model da vidimo da li ti isečci sadrže sliku. Kako?

Starije rjesenje je sliding window: pomeramo „prozor“ preko slike da selektujemo isečak. Za isečak izračunamo HOG deskriptor i primenimo trenirani model da vidimo da li isečak sadrži objekat ili ne.

Problemi: objekat se može nalaziti bilo gde na slici - moramo da analiziramo sve moguće isečke; objekti mogu biti različite veličine (udaljenost od kamere) - moramo ponoviti postupak na različitim skaliranim verzijama slike tj kako znati koja velicina isjecka da nam bude; overlapping bounding box problem = dobili smo više isečaka na kojima je (korektno) prepoznato lice, ali svi se odnose na isto lice -> ovo se rešava primenom nekog non-maximum suppression algoritma.

Alternativa sliding window pristupu je Region Proposal Algorithms...

- **Neuronske mreze**

Neuronska mreža je računarski model inspirisan strukturom i funkcionisanjem ljudskog mozga. Sastoji se od međusobno povezanih čvorova, često organizovanih u slojeve, koji rade zajedno na procesuiranju informacija i pravljenju predikcija.

Neuroni su osnovni gradivni blokovi neuronske mreže. Svaki neuron prima jedan ili više ulaza, obrađuje ih koristeći specifičnu funkciju aktivacije i proizvodi izlaz. Izlaz iz jednog neurona može poslužiti kao ulaz za drugi.

Neuronske mreže su organizovane u slojeve. Tri glavne vrste slojeva su:

Ulazni sloj: prima početne ulazne podatke. Cesto se ne broji kao sloj.

Skriveni slojevi: obrađuju ulazne podatke preko aktivacionih funkcija. Više skrivenih slojeva omogućava mreži da nauči složene obrasce.

Izlazni sloj: proizvodi konačni izlaz ili predviđanje. Broj neurona = broj kategorija. Ovdje se ne koriste aktivacione funkcije.

Postoje različite vrste neuronskih mreza:

Feedforward Neural Networks (FNN): neuronske mreže gde informacije teku u jednom pravcu, od ulaza do izlaza. Kaze chatgpt da su slojevi uvijek medjusobno potpuno povezani (fully connected layers).

Rekurentne neuronske mreže (RNN): mreže sa vezama koje formiraju cikluse, omogućavajući im da obrađuju nizove podataka.

Konvolucione neuronske mreže (CNN): Specijalizovane za zadatke obrade slika, koristeći konvolucione slojeve za ekstrakciju obeležja.

- **Funkcija greske/gubitka**

Koristi se tokom treniranja. Za ulaz x treba da izracunamo predikciju h(x) -> nn pronalazi tezine (w) takve da se predikcija h(x) sto bolje poklapa sa stvarnom vrijednoscu izlaza y. Kako nn pronalazi te tezine?

Racuna funkciju greske = razlika (obicno kvadratna = MSE) stvarne vrijednosti i predikcije naseg modela (zavisi od parametra w) i onda minimizujemo gresku po w tj zelimo da nadjemo w za koje je E(w) minimalno. Kako minimizovati funkciju greske? Pomocu gradijentnog spusta.

- **Gradijenti spust = Gradient Descent**

Cilj: minimizovati fju gubitka. Gradijent je izvod funkcije i daje pravac pomjeranja u kojem se greska lokalno najbrze povecava. Iterativno se pomjeramo ka minimumu. Azuriramo w u pravcu negativnog gradijenta greske E(w).

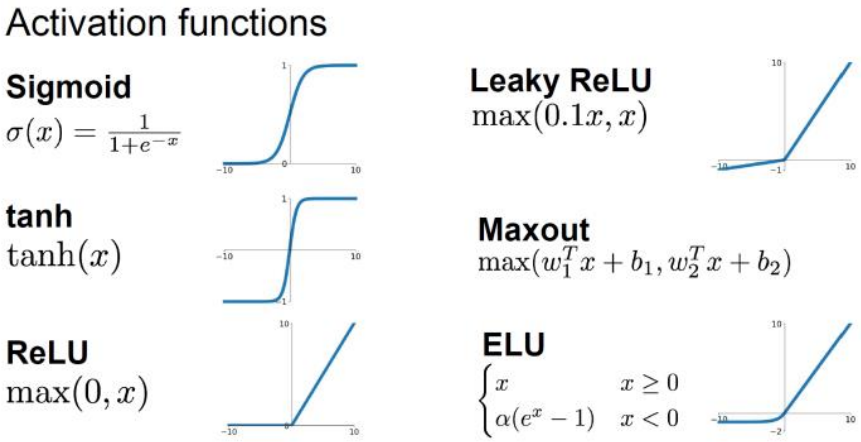
Backpropagation = algoritam propagacije u nazad: rekurzivna primjena pravila pronalaska izvoda složene funkcije. Nadjemo za posljedni sloj i dobijemo w i proslijedimo prethodnom sloju...

Learning rate = koliko brzo da uci tj koliko velike korake da pravi pri spustanju do minimuma (prevelik korak moze dovesti do zaobilazenja minimuma)

Momentum = da se ocuva direkcija, stvara otpornost na lokalne minimume i brze konvergira. Definise koliko se oslanja na prethodni racun (a koliko uvodi faktor nasumicnosti).

- **Aktivacione funkcije**

Aktivacione funkcije su ključni elementi u neuronskim mrežama koji uvode nelinearnost u model, omogućavajući mu da nauči kompleksne obrasce. Aktivaciona funkcjia transformise neki ulaz u neki izlaz, tj odredjuje nivo pobudjenosti neurona za date ulaze. Ulazima (x1, x2, ...) se pridruzuju tezine (w1, w2, ...) to se sabira i dodaje im se bias i onda aktivaciona funkcija transformise tu sumu. Razlicite aktivacione funkcije: sigmoidna, ReLU, leaky ReLU, tanh, maxout...



Sigmoidna: svodi prethodno navedenu sumu na interval [0, 1] tj normalizuje je. Problemi: eksponencijalna fja je skupa za racunanje, nije centrirana oko nule, ubija gradijente saturisanih neurona (vanishing gradients).

Tanh: mapira realan broj u opseg [−1,1]. Za razliku od sigmoida je centriran oko nule, ali kao i kod sigmoida dolazi do saturacije aktivacija.

ReLU: u praksi mnogo brže konvergira od sigmoida i tahn, brz, popularan. Neće saturisati u pozitivnoj regiji, ali problem za negativnu regiju, nije centriran oko nule = izlaz je uvijek nenegativan. Problem “umirucih” neurona ako neuroni postanu neaktivni tokom treninga.

Leaky ReLU: konvergira brze od sigmoida i tanh, efikasno izracunava. Ne dolazi do saturacije, neuroni ne “umiru”.

- **CNN**

CNN (Convolutional Neural Network), ili konvolutivna neuronska mreža, je vrsta neuronske mreže posebno dizajnirana za obradu i analizu vizuelnih podataka, kao što su slike. U okviru arhitekture CNN-a razlikujemo: konvolucione slojeve, pooling slojeve, i fully connected slojeve.

CNN vode racuna o topologiji tj prostornoj povezanosti slike (sto obicna NN ne radi). Svaki neuron gleda cijelu ulaznu sliku.

Konvolucijski sloj: ovo nije potpuno povezani sloj (glavna razlika je zadrzavanje prostore strukture). Konvolucija uključuje klizanje filtera (kernela) preko ulaza da bi se otkrili obrasci ili karakteristike. Ovi filteri mogu naučiti da prepoznaju ivice, teksture i karakteristike višeg nivoa. Ne definisemo mi sam kernel/filter (kao do sada) nego to CNN radi (ne zadajemo dubinu ni tezine nego samo visinu i sirinu i stride). Mozemo da koristimo vise filtera: svaki ce da rezultuje jednom aktivacionom mapom tj. svaki traži određeni koncept (šablon) u ulazima. Poslije konvolucije propustamo kroz aktivacionu funkciju (ReLU), pa tek onda ponovo u novi konvolucioni sloj ili pooling sloj.

Zero padding se dodaje da bismo umanjili smanjenje velicine aktivacionih mapa. Cilj je da veličina izlaza bude jednaka veličini ulaza. Bez zero padding, veličina aktivacionih mapa bi se brzo smanjivala sa brojem slojeva, što je nezgodno ako imamo zaista duboke mreže. Funkcionise tako sto se dodaju nule (u okviru matrice slike).

Pooling sloj: ovdje se obicno ne primjenjuju aktivacione funkcije. Ovi slojevi se koriste da bi se smanjila dimenzionalnost podataka, ocuvavajuci bitne informacije. Ne mijenja dubinu. Najcesci tipovi su Max Pooling i Average Pooling (to znas objasniti).

Potpuno povezani sloj (dense sloj): nakon nekoliko konvolucijskih i pooling slojeva, slijede jedan ili više potpuno povezanih slojeva za pravljenje predviđanja ili klasifikacija. Potpuno povezani slojevi agregiraju informacije iz ekstrahovanih karakteristika prethodnih slojeva i generišu konačni izlaz. Ovdje se isto koriste aktivacione funkcije. Definicija poptuno povezanog sloja.

- **ANN**

ANN (Artificial Neural Network), ili veštačka neuronska mreža, je model inspirisan biološkim neuronima. U sutini sve isto kao za NN: 3 sloja (ulazni, skriveni, izlazni), svi slojevi su poptuno povezani, koriste kombinaciju feedforward i backpropagation, ...

- **Razlika CNN & ANN**

1. slojevi: podjela na slojeve im se razlikuje

2. topologija: ann ne uzima u obzir topologiju tj prostorne podatke dok cnn uzima

3. povezanost: ann svi slojevi su fully-connected, a kod cnn konvolucijski i pooling nisu

4. parametri: ann: svaka konekcija između neurona ima svoj skup težina i nema dijeljenja parametara. Težine se uče nezavisno za svaku konekciju.

Cnn: koriste dijeljenje parametara u konvolucionim slojevima, gdje se isti skup težina filtera primjenjuje na različite prostorne lokacije. Ovo pomaže u hvatanju lokalnih obrazaca i smanjuje broj parametara.

5. hijerarhija: ann procesira podatke globalno bez obzira na hijerarhiju/dubinu mreze, dok cnn na nižim slojevima obično snima jednostavne karakteristike kao što su ivice, dok viši slojevi snimaju apstraktnije i složenije karakteristike.

6. primjena: cnn bas za slike, detekciju objekata, segmentaciju itd. Ann za regresiju, klasifikaciju...

- **Metrike performansi**

Metrike performansi, poznate i kao metrike evaluacije ili metrike procene, su kvantitativne mere koje se koriste za procenu performansi modela, sistema ili procesa.

TP (true positive) = instance koje pripadaju pozitivnoj klasi i koje ispravno predviđene kao pozitivne, FP (false positive) = instance koje pripadaju negativnoj klasi, ali su pogrešno predviđene kao pozitivne, TN (true negative) = instance koje pripadaju negativnoj klasi i koje su ispravno predviđene kao negativne, FN (false negative) = instance koje pripadaju pozitivnoj klasi i koje pogresno predviđene kao negativne

1. Tacnost (Accuracy) = udeo tačno klasifikovanih instanci u odnosu na ukupan broj instanci. $\frac{\mathrm{TP}+\mathrm{TN}}{\mathrm{TP}+\mathrm{TN}+\mathrm{FP}+\mathrm{FN}}$
2. Preciznost = mjeri tacnost pozitivnih predikcija. Odnosno koliko je ono sto je on oznacio tacnim zaista tacno $\frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}}$. Preciznost je osjetljiva na FP.

3. Odziv (Recall) = mjeri sposobnost modela da identifikuje sve stvarne pozitivne instance $\frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}}$ Odziv je osjetljiv na FN.

   Odziv je posebno važan u situacijama kada je bitno identifikovati sve stvarno pozitivne instance i minimizovati lažno negativne. Na primer, u medicinskim dijagnostičkim sistemima, gde je otkrivanje stvarno pozitivnih slučajeva (bolesti) ključno, odziv može biti kritičan. Visok odziv ukazuje na to da model dobro prepoznaje stvarno pozitivne slučajeve, ali može rezultirati većim brojem lažno pozitivnih slučajeva.

4. F1-mera = Harmonična sredina preciznosti i odziva, korisna kada želimo balans između ove dve metrike. $2 \times \frac{\mathrm{Precision} \times \mathrm{Recall}}{\mathrm{Precision}+\mathrm{Recall}}$

- **Klasifikacija**

...